Okno ze zdjęcia – wersja 3D (Three.js)

Plan technologii i kolejnych kroków · listopad 2026

1. Pomysł w jednym zdaniu

Użytkownik wrzuca dowolne zdjęcie, a aplikacja – dzięki AI, które rozpoznaje głębię i niebo – buduje z niego **prawdziwą scenę 3D** (wypukłą siatkę), po której porusza się kamera razem z głową widza. Z kamerką i projekcją off-axis ekran zachowuje się jak prawdziwe okno.

2. Jak działa aplikacja (oczami użytkownika)

1. Użytkownik klika **„Zrób okno z własnego zdjęcia”** i wrzuca zwykłe zdjęcie (JPG/PNG).
2. Aplikacja pokazuje „Analizuję zdjęcie…” z paskiem postępu. W tym czasie w przeglądarce działają modele AI.
3. **AI nr 1 – głębia** (Depth Anything V2): dla każdego piksela ocenia, jak daleko jest od aparatu.
4. **AI nr 2 – niebo** (segmentacja SegFormer): zaznacza, gdzie na zdjęciu jest niebo.
5. Zdjęcie zamienia się w okno: ruch myszki / przechył telefonu / **ruch głowy** (AI nr 3 – MediaPipe, wykrywanie twarzy z kamerki) przesuwa widok z efektem głębi.
6. Niebo ze zdjęcia jest podmienione na animowane niebo z projektu: działa suwak pory dnia (zachód, noc, gwiazdy) i pogoda.
7. Opcjonalnie **AI nr 4 – Claude** opisuje zdjęcie i dobiera dźwięki oraz klimat (jedyna płatna część).

3. Co dokładnie robi AI

AI	Wejście	Wyjście	Do czego	Koszt
Depth Anything V2 (small)	zdjęcie	mapa głębi (szara: jasne = blisko)	efekt głębi przy ruchu	0 zł, w przeglądarce
SegFormer (ADE20K, klasa „sky”)	zdjęcie	maska nieba	podmiana nieba → dzień/noc, pogoda	0 zł, w przeglądarce
MediaPipe Face Landmarker	obraz z kamerki	pozycja głowy	sterowanie widokiem głową	0 zł, w przeglądarce
Claude API (opcjonalnie)	zdjęcie	opis + dobór dźwięków/klimatu	dopasowanie atmosfery	płatne, grosze/zapytanie, wymaga backendu

Cała reszta – budowa siatki 3D, shadery, kamera off-axis, niebo, pogoda, UI – to **wasz własny kod**.

4. Technologie

Warstwa	Technologia	Rola
Render 3D	three (r187, już w package.json)	PlaneGeometry z gęstą siatką, własny ShaderMaterial, PerspectiveCamera
Shadery	GLSL (ShaderMaterial)	Przesunięcie wierzchołków wg głębi, odrzucanie rozciągniętych trójkątów, dzień/noc
Niebo	Sky z three/addons lub własny shader	Animowane niebo za siatką zdjęcia
Pogoda	THREE.Points / InstancedMesh	Cząsteczki na prawdziwej głębokości (bufor głębi robi zasłanianie)
AI w przeglądarce	@huggingface/transformers	Depth Anything + SegFormer (ONNX, WebGPU/WASM)

Warstwa	Technologia	Rola
Kamera	@mediapipe/tasks-vision	Face Landmarker – pozycja głowy, także odległość od ekranu
Wątek w tle	Web Worker	Modele AI poza wątkiem UI
Język / build / testy	TypeScript strict, Vite, Vitest	Bez zmian
Opcjonalnie	Claude API + Cloudflare Worker	Opis zdjęcia i dobór klimatu

CLAUDE.md obecnie ZABRANIA Three.js („Bez silnika gier i bez Three.js"). Przed rozpoczęciem trzeba zmienić tę zasadę (decyzja zespołu + najlepiej zgoda prowadzącego) i zdecydować, czy PixiJS zostaje w projekcie.

5. Jak to działa technicznie

Zdjęcie jako siatka 3D

Płaszczyzna dzielona jest na np. 256×256 kwadratów. Vertex shader odczytuje mapę głębi i odsuwa każdy wierzchołek w głąb sceny. Zdjęcie jest nałożone jako tekstura – powstaje „płaskorzeźba" ze zdjęcia.

```
// vertex shader (sketch)
uniform sampler2D depthMap;
uniform float depthScale;
varying vec2 vUv;
void main() {
  vUv = uv;
  float d = texture2D(depthMap, uv).r;          // 1 = near, 0 = far
  vec3 p = position;
  p.z -= (1.0 - d) * depthScale;                 // push far pixels back
  gl_Position = projectionMatrix * modelViewMatrix * vec4(p, 1.0);
}
```

Krawędzie obiektów

Na granicy drzewo/tło trójkąty się rozciągają. Shader wykrywa duży skok głębi i odrzuca te fragmenty (discard). Powstałe dziury zasłania druga, dalsza warstwa z rozmytym tłem (prosty „inpainting" bez AI).

Prawdziwe okno: projekcja off-axis

Zwykła kamera obraca się wokół sceny. W oknie jest inaczej: ekran to stała „szyba", a zmienia się położenie oka. Pozycja głowy z MediaPipe (x, y oraz odległość z rozmiaru twarzy) ustawia kamerę, a macierz projekcji jest liczona asymetrycznie, tak by krawędzie ekranu zawsze były krawędziami okna:

```
camera.position.set(head.x, head.y, head.z);
camera.projectionMatrix.makePerspective(left, right, top, bottom, near, far);
// left/right/top/bottom computed from screen corners relative to the head (pure function, Vitest)
```

Pogoda i niebo

Deszcz i śnieg to cząsteczki w przestrzeni 3D – bufor głębi sam sprawia, że krople są przed górami, a za bliskimi kamieniami. Niebo z maski segmentacji jest przezroczyste, za nim stoi animowane niebo (Sky + TimeOfDay).

Proponowane pliki

```
src/ai/depthWorker.ts          # Web Worker: models
src/ai/depth.ts, sky.ts        # API for depth map and sky mask
src/ai/depthUtils.ts           # pure functions (Vitest)
src/three/photo/DepthMesh.ts   # PlaneGeometry + ShaderMaterial
src/three/photo/shaders/*.glsl # vertex/fragment shaders
src/three/camera/OffAxis.ts    # off-axis projection (math in utils/, tested)
src/three/effects/Weather3D.ts # particles in 3D
src/ui/PhotoUpload.ts          # upload, progress, errors
```

6. Co robić dalej – kroki

Krok 0 – Decyzja o silniku (1 spotkanie)

- Zdecydujcie: Three.js zamiast PixiJS czy obok niego? Zaktualizujcie zasady w CLAUDE.md.
- Zatwierdźcie zależność @huggingface/transformers.

» **Gotowe, gdy:** CLAUDE.md nie zabrania już Three.js; jasny podział silników.

Krok 1 – Prototyp głębi (1–2 dni)

- Strona testowa: zdjęcie → mapa głębi; test na 5–10 waszych zdjęciach.

» **Gotowe, gdy:** widać sensowną mapę głębi.

Krok 2 – Siatka 3D (2–3 dni)

- DepthMesh.ts: PlaneGeometry 256×256 + vertex shader; kamera sterowana myszką.
- Dobór depthScale, wygładzenie mapy głębi.

» **Gotowe, gdy:** zdjęcie wygląda jak płaskorzeźba, ruch myszką pokazuje głębię.

Krok 3 – Krawędzie (2–3 dni)

- Wykrywanie skoków głębi w shaderze, discard, warstwa tła wypełniająca dziury.

» **Gotowe, gdy:** brak mocno rozciągniętych trójkątów przy umiarkowanym ruchu.

Krok 4 – Web Worker + UI (1–2 dni)

- Model w Workerze, pasek postępu, obsługa błędów i fallback WASM.

» **Gotowe, gdy:** UI płynne, błędy nie wywalają aplikacji.

Krok 5 – Kamera i off-axis (3–4 dni)

- CameraInput (MediaPipe), wygładzanie, kalibracja „usiądź na środku”.
- OffAxis.ts: czysta funkcja liczenia frustum + testy Vitest.

» **Gotowe, gdy:** ruch głową daje efekt prawdziwego okna (najmocniejszy punkt demo).

Krok 6 – Niebo, dzień/noc, pogoda (3–4 dni)

- Maska nieba, Sky/TimeOfDay, cząsteczki 3D, mgła (scene.fog).

» **Gotowe, gdy:** suwak pory dnia i pogoda działają na dowolnym zdjęciu.

Krok 7 – Opcjonalnie Claude (2 dni)

- Cloudflare Worker z kluczem, endpoint „opisz zdjęcie → JSON klimatu”.

» **Gotowe, gdy:** atmosfera dobiera się automatycznie.

Krok 8 – Wydajność, testy, prezentacja (2–3 dni)

- Mniejsza siatka na telefonach (128×128), pixelRatio max 2, testy iOS/Android.
- Filmik z ruchem głowy na prezentację.

» **Gotowe, gdy:** płynnie na telefonie średniej klasy; build, lint, test przechodzą.

7. Ryzyka i ograniczenia

- **Rozmiar modeli:** pierwsze pobranie to kilkadziesiąt MB (potem cache przeglądarki). Ładować dopiero po kliknięciu „wrzuć zdjęcie”.
- **Czas analizy:** komputer ok. 1–3 s, telefon ok. 5–15 s. Robione raz na zdjęcie, w Web Workerze (UI się nie zacina).
- **Trudne zdjęcia:** szkło, woda, mgła, drobne gałęzie, włosy – mapa głębi bywa błędna. Płaskie zdjęcia (ściana) dają słaby efekt.
- **Klucz Claude API** nigdy w kodzie frontendu (GitHub Pages jest publiczne) – tylko przez mały backend (np. Cloudflare Worker).

- **Nazwy modeli i API bibliotek** w tym dokumencie należy zweryfikować w aktualnej dokumentacji przed implementacją.
- **Specyficzne dla 3D:** więcej nowej wiedzy (shadery GLSL, macierze projekcji) – trzeba ją rozumieć na obronie.
- **Wydajność:** siatka 256×256 to ok. 130 tys. trójkątów – na słabych telefonach zmniejszyć gęstość.
- **Dwa silniki w projekcie** (PixiJS + Three.js) to więcej kodu i większa paczka – lepiej wybrać jeden.

8. Podsumowanie

	Wersja 3D (Three.js)
Trudność	średnia–wysoka
Jakość efektu	wyraźnie lepsza; z kamerką – prawdziwe okno
Zgodność z CLAUDE.md	nie – wymaga zmiany zasad
Koszt	0 zł (bez opcjonalnego Claude)
Szacowany czas	ok. 3–5 tygodni pracy w dwie osoby